Baizhuma Aibek

HIGH-PERFORMANCE & PARALLEL COMPUTING

Lab 1

```
================ RUN 1 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 51896
[Worker] Logical Rank: 3 of 4 | Native OS TID: 7704
[Worker] Logical Rank: 1 of 4 | Native OS TID: 51896
[Worker] Logical Rank: 2 of 4 | Native OS TID: 54036
--- Joined thread team. Execution returned to serial master ---

================ RUN 2 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 54180
[Worker] Logical Rank: 3 of 4 | Native OS TID: 45008
[Worker] Logical Rank: 1 of 4 | Native OS TID: 54180
[Worker] Logical Rank: 2 of 4 | Native OS TID: 44844
--- Joined thread team. Execution returned to serial master ---

================ RUN 3 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 48948
[Worker] Logical Rank: 3 of 4 | Native OS TID: 45604
[Worker] Logical Rank: 1 of 4 | Native OS TID: 48948
[Worker] Logical Rank: 2 of 4 | Native OS TID: 37352
--- Joined thread team. Execution returned to serial master ---

================ RUN 4 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 52168
[Worker] Logical Rank: 3 of 4 | Native OS TID: 5532
[Worker] Logical Rank: 1 of 4 | Native OS TID: 52168
[Worker] Logical Rank: 2 of 4 | Native OS TID: 46576
--- Joined thread team. Execution returned to serial master ---

================ RUN 5 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 25816
[Worker] Logical Rank: 3 of 4 | Native OS TID: 50260
[Worker] Logical Rank: 1 of 4 | Native OS TID: 25816
[Worker] Logical Rank: 2 of 4 | Native OS TID: 8576
--- Joined thread team. Execution returned to serial master ---

================ RUN 6 ================
```

```
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 54076
[Worker] Logical Rank: 3 of 4 | Native OS TID: 53608
[Worker] Logical Rank: 1 of 4 | Native OS TID: 54076
[Worker] Logical Rank: 2 of 4 | Native OS TID: 36624
--- Joined thread team. Execution returned to serial master ---

================ RUN 7 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 42732
[Worker] Logical Rank: 3 of 4 | Native OS TID: 43932
[Worker] Logical Rank: 1 of 4 | Native OS TID: 42732
[Worker] Logical Rank: 2 of 4 | Native OS TID: 17488
--- Joined thread team. Execution returned to serial master ---

================ RUN 8 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 50212
[Worker] Logical Rank: 3 of 4 | Native OS TID: 38500
[Worker] Logical Rank: 1 of 4 | Native OS TID: 50212
[Worker] Logical Rank: 2 of 4 | Native OS TID: 13836
--- Joined thread team. Execution returned to serial master ---

================ RUN 9 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 21540
[Worker] Logical Rank: 3 of 4 | Native OS TID: 52868
[Worker] Logical Rank: 1 of 4 | Native OS TID: 21540
[Worker] Logical Rank: 2 of 4 | Native OS TID: 19284
--- Joined thread team. Execution returned to serial master ---

================ RUN 10 ================
--- Forking a team of 4 threads ---
[Master] Logical Rank: 0 of 4 | Native OS TID: 44704
[Worker] Logical Rank: 3 of 4 | Native OS TID: 33548
[Worker] Logical Rank: 1 of 4 | Native OS TID: 44704
[Worker] Logical Rank: 2 of 4 | Native OS TID: 35740
--- Joined thread team. Execution returned to serial master ---
```

The consistent ordering in this particular experiment was also influenced by the artificial delays in the starter implementation. Rank 0 and rank 3 receive no sleep delay, while rank 1 sleeps for approximately 1 ms and rank 2 for approximately 2 ms.

	Threads	Execution_Time_Seconds
1	1	0.001956599997356534
2	2	0.0008048000163398683
3	4	0.0009458999847993255
4	8	0.001792500028386712
5	16	0.0026789000257849693
6	32	0.008023599977605045
7	64	0.012925099988933653

The test computer contains 10 physical CPU cores and 12 logical processors. Therefore, configurations such as 16, 32, and 64 software threads exceed the available hardware execution capacity.

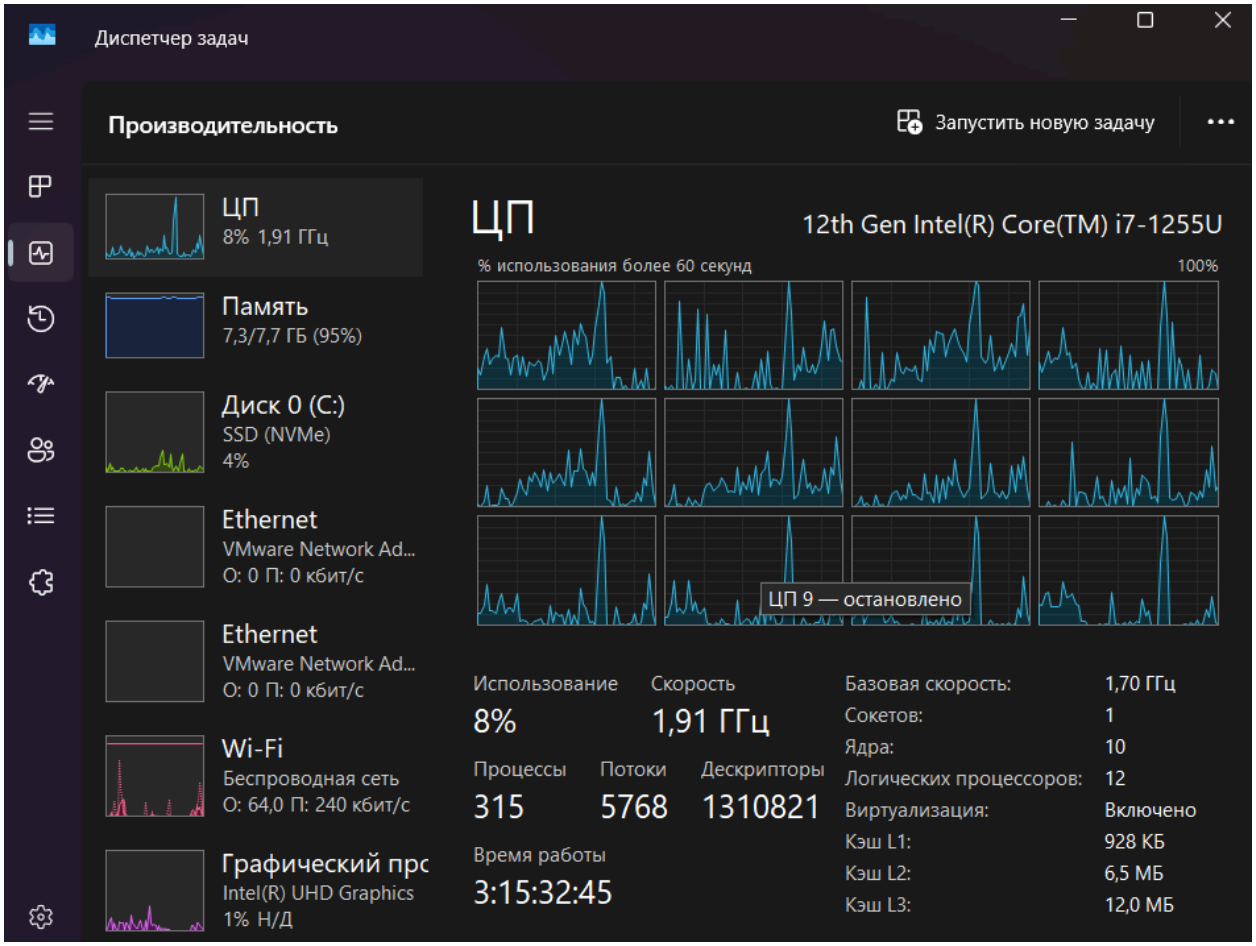
```
(.venv) PS C:\Dev\PythonProjects> python .\lab1_task1_3.py
Task 1.3 - CPU Core Saturation Analysis
----------------------------------------
Logical processors detected: 12
Threads used: 12
Square roots per thread: 500,000,000

Warming up Numba...
Starting CPU workload...

Workload completed.
Execution Time: 2.4712 seconds
Result check: 7453559936179.85
```

Threads: 12
Iterations per thread: 500,000,000
Execution Time: 2.4712 seconds
Result Check: 7453559936179.85

The number of threads was selected based on the 12 logical processors detected on the test system.



The workload was increased beyond the example 10,000,000 operations in order to make the execution interval long enough for CPU utilization to be observed in Task Manager.

System Configuration:
CPU: 12th Gen Intel Core i7-1255U
Physical Cores: 10
Logical Processors: 12
Base Clock: 1.70 GHz

L1 Cache: 928 KB
L2 Cache: 6.5 MB
L3 Cache: 12.0 MB

Operating System: Windows
Language: Python

Question 1.1

In this experiment, all ten executions produced the logical order 0, 3, 1, 2. This stable pattern was strongly influenced by the artificial sleep delays in the starter program. Rank 0 and rank 3 have no delay, rank 1 waits approximately 1 ms, and rank 2 waits approximately 2 ms.

Question 1.2

Oversubscription occurs when the number of runnable software threads exceeds the available hardware processing resources.

The test system contains 10 physical cores and 12 logical processors. Therefore, running 16, 32, or 64 software threads requires multiple threads to compete for the available CPU execution contexts.

The operating system uses context switching to share CPU resources between runnable threads. During a context switch, the execution state of one thread must be saved and another restored. Excessive switching creates scheduling overhead.

Large numbers of competing threads may also reduce cache efficiency because the working data of different threads competes for limited cache capacity. This can increase cache misses and cache thrashing.

The experimental results support this behavior: execution time increased from approximately 0.002679 s with 16 threads to 0.008024 s with 32 threads and 0.012925 s with 64 threads.

Question 1.3

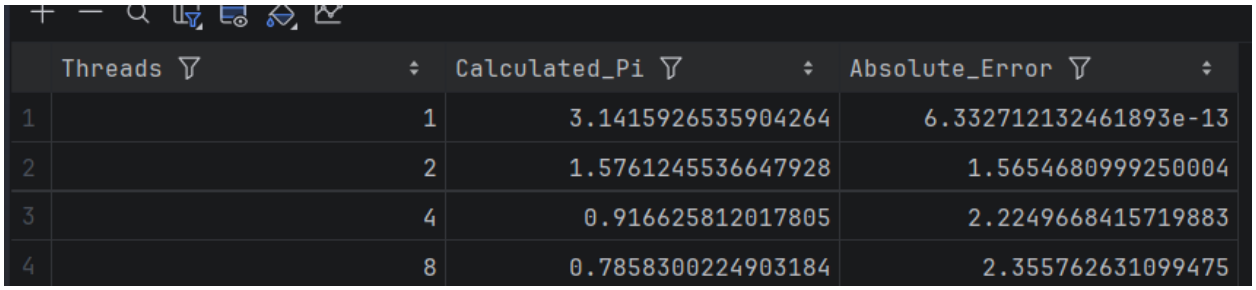
An implicit barrier at the end of an OpenMP parallel region ensures that all worker threads complete their assigned operations before program execution continues beyond the parallel section.

Without this synchronization, the master thread or another worker could continue executing code that depends on results that have not yet been completed.

This could lead to incomplete data being read, shared data being modified prematurely, resources being released while other threads still use them, or other race-condition-related problems.

Therefore, the barrier provides a synchronization point that guarantees completion of the parallel region before subsequent serial execution continues.

Lab 2

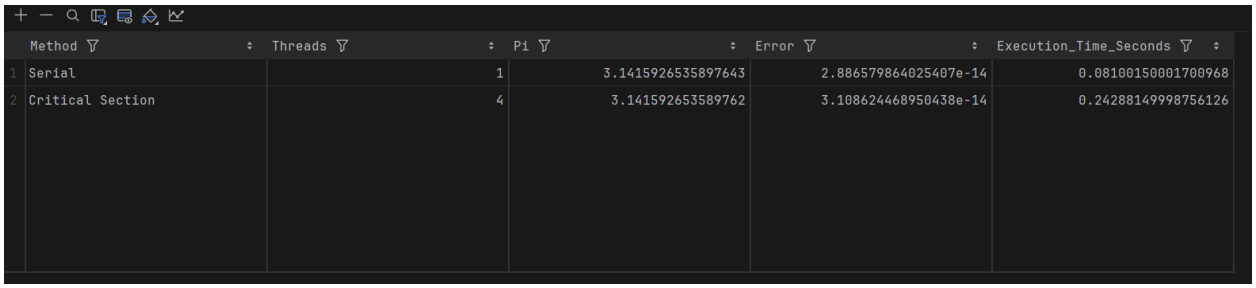


	Threads	Calculated_Pi	Absolute_Error
1	1	3.1415926535904264	6.332712132461893e-13
2	2	1.5761245536647928	1.5654680999250004
3	4	0.916625812017805	2.2249668415719883
4	8	0.7858300224903184	2.355762631099475

The naive shared accumulator was executed using 1, 2, 4, and 8 threads. With one thread, the calculated value of Pi was 3.141592653590 with an absolute error of approximately 6.33×10^{-13}. When multiple threads were used, the numerical error increased significantly. With 8 threads, the calculated value decreased to approximately 0.785830, producing an absolute error of about 2.355763. This behavior occurs because multiple threads perform unsynchronized read-modify-write operations on the same shared accumulator. As a result, some updates overwrite other updates, causing lost contributions to the final sum.

```
(.venv) PS C:\Dev\PythonProjects> python .\lab2_task2_1.py
Lab 2 - Task 2.1
Race Condition Quantification
----------------------------------------------
N = 100,000,000

Threads: 1 | Pi: 3.141592653590 | Absolute Error: 6.332712132462e-13
Threads: 2 | Pi: 1.576124553665 | Absolute Error: 1.565468099925e+00
Threads: 4 | Pi: 0.916625812018 | Absolute Error: 2.224966841572e+00
Threads: 8 | Pi: 0.785830022490 | Absolute Error: 2.355762631099e+00
```



	Method	Threads	Pi	Error	Execution_Time_Seconds
1	Serial	1	3.1415926535897643	2.886579864025407e-14	0.08100150001700968
2	Critical Section	4	3.141592653589762	3.108624468950438e-14	0.24288149998756126

```
(.venv) PS C:\Dev\PythonProjects> python .\lab2_task2_2.py
Lab 2 - Task 2.2
Critical Section Overhead
----------------------------------------
N = 1,000,000
Critical Section Threads = 4

Serial Pi: 3.141592653590 | Error: 2.886579864025e-14
Serial Time: 0.081002 seconds

Critical Pi: 3.141592653590 | Error: 3.108624468950e-14
Critical Time: 0.242881 seconds

Lock Contention Overhead: 199.85%
```

The serial implementation completed the numerical integration in 0.081002 seconds, while the four-thread critical-section implementation required 0.242881 seconds. Both implementations produced an accurate approximation of Pi, demonstrating that mutual exclusion successfully eliminated the race condition observed in Task 2.1. However, the critical-section implementation produced a lock contention overhead of 199.85%. This occurs because only one thread can update the shared accumulator at a time, forcing the other threads to wait and effectively serializing the most frequently executed part of the parallel computation.

	Threads	Trial	Execution_Time_Seco…	Calculated_Pi	Absolute_Error
1	1	1	0.15772580000339076	3.1415926535904264	6.332712132461893e-13
2	1	2	0.14069540001219139	3.1415926535904264	6.332712132461893e-13
3	1	3	0.14407700003357604	3.1415926535904264	6.332712132461893e-13
4	1	4	0.14334940002299845	3.1415926535904264	6.332712132461893e-13
5	1	5	0.14431390003301203	3.1415926535904264	6.332712132461893e-13
6	2	1	0.07912790001137182	3.14159265358991	1.1679546219056647e-13
7	2	2	0.0770516999764368	3.14159265358991	1.1679546219056647e-13
8	2	3	0.07925000000977889	3.14159265358991	1.1679546219056647e-13
9	2	4	0.07717020000563934	3.14159265358991	1.1679546219056647e-13
10	2	5	0.11171519994968548	3.14159265358991	1.1679546219056647e-13
11	4	1	0.06510840001283213	3.1415926535896825	1.1057821325266559e-13
12	4	2	0.061938000028021634	3.1415926535896825	1.1057821325266559e-13
13	4	3	0.06681500002741814	3.1415926535896825	1.1057821325266559e-13
14	4	4	0.06599669996649027	3.1415926535896825	1.1057821325266559e-13
15	4	5	0.062040099990554154	3.1415926535896825	1.1057821325266559e-13
16	8	1	0.040661900013219565	3.141592653589815	2.1760371282653068e-14
17	8	2	0.04039270000066608	3.141592653589815	2.1760371282653068e-14
18	8	3	0.040270600002259016	3.141592653589815	2.1760371282653068e-14
19	8	4	0.04035290004685521	3.141592653589815	2.1760371282653068e-14
20	8	5	0.040540300018619746	3.141592653589815	2.1760371282653068e-14
21	16	1	0.04183870000997558	3.1415926535898824	8.926193117986259e-14
22	16	2	0.04051880002953112	3.1415926535898824	8.926193117986259e-14
23	16	3	0.040406200004525408	3.1415926535898824	8.926193117986259e-14

	Threads	Average_Time_Secon…	Calculated_Pi	Absolute_Error
1	1	0.14603230002103373	3.1415926535904264	6.332712132461893e-13
2	2	0.08486299999058247	3.14159265358991	1.1679546219056647e-13
3	4	0.06437964000506327	3.1415926535896825	1.1057821325266559e-13
4	8	0.040443680016323924	3.141592653589815	2.1760371282653068e-14
5	16	0.041269100015051664	3.1415926535898824	8.926193117986259e-14

The parallel reduction achieved a speedup of approximately 1.95× with 2 threads, 2.40× with 4 threads, and 3.57× with 8 threads. Increasing the number of threads from 8 to 16 produced almost no additional performance improvement, with speedup increasing only from 3.57× to 3.59×. Parallel efficiency therefore decreased as thread count increased, from 97.73% at 2 threads to 22.45% at 16 threads. This indicates that synchronization overhead, scheduling overhead, memory-system limitations, and the finite number of hardware execution contexts prevent linear scaling.

Question 2.1

A non-atomic read-modify-write operation usually consists of several separate steps: first the CPU loads the current value from memory or cache into a register, then performs an arithmetic operation, and finally stores the new value back.

When several threads execute these operations on the same shared variable at the same time, they may read the same old value before either thread writes its result. Both threads then calculate a new value independently and write it back. One update can overwrite another, producing a lost update.

This is what happened in the Naive Race implementation. With one thread, the calculated Pi value was almost exact, but as the thread count increased, more updates were lost and the numerical error became much larger.

Question 2.2

A centralized critical section allows only one thread at a time to update the shared accumulator. Therefore, as the number of threads increases, more threads must wait for the same lock, creating a serialization bottleneck.

A binary reduction tree works differently. Each thread first computes its own local partial result. These independent partial results are then combined pairwise in multiple stages.

Because the number of active values is approximately halved at each stage, the number of combination stages grows as O(log P) rather than O(P).

Therefore, parallel reduction scales much better than a centralized critical section because most of the computation can proceed independently without continuous lock contention.

This is consistent with our experiment: the critical-section version took about 0.242881 seconds, while the reduction implementation achieved significantly better scaling across multiple threads

Question 2.3

In practice, the measured speedup usually becomes flat much earlier. In our experiment, performance improved significantly up to 8 threads, but increasing from 8 to 16 threads produced almost no additional improvement.

Several architectural factors can cause this, including thread scheduling overhead, synchronization costs, memory bandwidth limitations, cache contention, communication between CPU cores, and oversubscription. Our system has 12 logical processors, so running 16 software threads also causes additional competition for hardware resources

Lab 3

```
(.venv) PS C:\Dev\PythonProjects> python .\lab3_task3_1.py
Lab 3 - Task 3.1
Static vs Dynamic Scheduling
----------------------------------------
Warming up Numba...
Warm-up completed.

STATIC
Threads: 4
Chunk Size: 16
Time: 0.2152 s
Work per thread: [51027542, 50414919, 49819435, 51854983]
Imbalance: 0.0401

DYNAMIC
Threads: 4
Chunk Size: 16
Time: 0.1937 s
Work per thread: [50782579, 51826240, 50629345, 49878715]
Imbalance: 0.0384
```

```
Threads=2, Chunk=1
 Trial 1: 0.3237s | Imbalance: 0.0012
 Trial 2: 0.3392s | Imbalance: 0.0012
 Trial 3: 0.3727s | Imbalance: 0.0012
 Average Time: 0.3452s
 Average Imbalance: 0.0012

Threads=2, Chunk=16
 Trial 1: 0.3194s | Imbalance: 0.0140
 Trial 2: 0.3276s | Imbalance: 0.0140
 Trial 3: 0.3204s | Imbalance: 0.0140
 Average Time: 0.3225s
 Average Imbalance: 0.0140

Threads=2, Chunk=64
 Trial 1: 0.3332s | Imbalance: 0.1048
 Trial 2: 0.3512s | Imbalance: 0.1048
 Trial 3: 0.3342s | Imbalance: 0.1048
 Average Time: 0.3396s
 Average Imbalance: 0.1048

Threads=2, Chunk=256
 Trial 1: 0.3940s | Imbalance: 0.4335
 Trial 2: 0.4111s | Imbalance: 0.4335
 Trial 3: 0.3946s | Imbalance: 0.4335
 Average Time: 0.3999s
 Average Imbalance: 0.4335

Threads=4, Chunk=1
 Trial 1: 0.1936s | Imbalance: 0.0038
 Trial 2: 0.1981s | Imbalance: 0.0038
 Trial 3: 0.1913s | Imbalance: 0.0038
 Average Time: 0.1943s
 Average Imbalance: 0.0038
```

	Scheduler	Threads	Chunk_Size	Average_Time_Seconds	Average_Imbalance
1	Static	2	1	0.34517793332148966	0.0012409308435661814
2	Static	2	16	0.3224505999824032	0.014010898621576397
3	Static	2	64	0.339552866644226	0.10477028844067655
4	Static	2	256	0.3999094666990762	0.43351895929830625
5	Static	4	1	0.19432719998682538	0.003801653529739397
6	Static	4	16	0.19855456665391102	0.04008624019867891
7	Static	4	64	0.19957399998869127	0.13095124408641587
8	Static	4	256	0.3947727333482665	2.3946522730885405
9	Static	8	1	0.11284633333949994	0.013123439140673286
10	Static	8	16	0.1140113666576023	0.16119700224421035
11	Static	8	64	0.17713520000688732	1.9271346523594428
12	Static	8	256	0.4040482999989763	4.828171232386847
13	Static	16	1	0.11124789998090516	0.018160814690343877
14	Static	16	16	0.11342673333516966	0.30953709169586047
15	Static	16	64	0.17990756665434068	3.941294903413714
16	Static	16	256	0.3910569333238527	9.656342464773694
17	Dynamic	2	1	0.3215336333378218	0.0062389333319101785
18	Dynamic	2	16	0.3179973999892051	0.0030390417069507385
19	Dynamic	2	64	0.3349089000063638	0.08520320952745636
20	Dynamic	2	256	0.39100863330531865	0.43076086256721186
21	Dynamic	4	1	0.19325650000246242	0.10919450306572831
22	Dynamic	4	16	0.19143756665289402	0.09082979919819135
23	Dynamic	4	64	0.20799323334358633	0.13368683817425794

	Scheduler	Threads	Chunk_Size	Trial	Execution_Time_Se…	Imbalance
1	Static	2	1	1	0.3236579999793321	0.0012409308435661814
2	Static	2	1	2	0.3391824999707751	0.0012409308435661814
3	Static	2	1	3	0.37269330001436174	0.0012409308435661814
4	Static	2	16	1	0.31939259998034686	0.014010898621576397
5	Static	2	16	2	0.3275804999866523	0.014010898621576397
6	Static	2	16	3	0.3203786999802105	0.014010898621576397
7	Static	2	64	1	0.3332281999755651	0.10477028844067657
8	Static	2	64	2	0.35120269999606535	0.10477028844067657
9	Static	2	64	3	0.3342276999610476	0.10477028844067657
10	Static	2	256	1	0.3940360000124201	0.43351895929830625
11	Static	2	256	2	0.4111092000384815	0.43351895929830625
12	Static	2	256	3	0.39458320004632697	0.43351895929830625
13	Static	4	1	1	0.19361119996756315	0.003801653529739397
14	Static	4	1	2	0.19809019996318966	0.003801653529739397
15	Static	4	1	3	0.19128020002972335	0.003801653529739397
16	Static	4	16	1	0.1985755999921821	0.04008624019867891
17	Static	4	16	2	0.1999665999901481	0.04008624019867891
18	Static	4	16	3	0.19712149997940287	0.04008624019867891
19	Static	4	64	1	0.20367409999016672	0.13095124408641587
20	Static	4	64	2	0.19712289999006316	0.13095124408641587
21	Static	4	64	3	0.1979249999858439	0.13095124408641587
22	Static	4	256	1	0.3897315000067465	2.3946522730885405
23	Static	4	256	2	0.39415940000020726	2.3946522730885405

Static Scheduling

In the Static scheduler, the work was divided into chunks before execution. Chunks were assigned to worker threads in a deterministic round-robin pattern.

Therefore, each worker thread received its work before computation began.

Dynamic Scheduling

In the Dynamic scheduler, the next available row chunk was stored in a shared work queue. After a thread completed its current chunk, it requested another chunk from the queue.

A lock was used when obtaining the next chunk to ensure that two worker threads could not claim the same work.

Question 3.1 — Fine-Grained Contention

Dynamic scheduling with a chunk size of 1 provides very fine-grained load balancing because each idle worker can immediately request one additional unit of work.

However, this also increases scheduling overhead. Every worker frequently accesses the shared work queue to obtain the next chunk.

In the implementation used in this experiment, the work queue is protected by synchronization. When many threads repeatedly request very small chunks, they compete for access to the shared queue and its synchronization mechanism.

Therefore, although chunk size = 1 may provide excellent load balancing, synchronization and queue-management overhead can reduce overall performance.

Question 3.2 — Spatial Imbalance

In Static scheduling, each worker receives a predetermined section of the Mandelbrot workload.

However, the computational cost of different areas of the Mandelbrot image is not uniform. Pixels outside the set often escape after only a small number of iterations, while pixels close to or inside the Mandelbrot boundary may require hundreds or all 1000 iterations.

Therefore, worker threads assigned to computationally dense regions can become stragglers. Other threads may finish their assigned sections earlier and remain idle while waiting for the slower threads to reach the synchronization point.

The exact thread rank that becomes a straggler depends on how rows and chunks are mapped to thread ranks.

Question 3.3 — Guided Scheduling

Guided scheduling combines characteristics of both Static and Dynamic scheduling.

At the beginning of execution, relatively large chunks of iterations are assigned to worker threads. This reduces scheduling overhead because workers do not frequently request new work.

As the number of remaining iterations decreases, the scheduler gradually decreases the chunk size.

Therefore, near the end of the workload, smaller chunks provide improved load balancing and reduce the probability that one worker receives a disproportionately expensive final portion of the computation.

Guided scheduling attempts to combine the lower scheduling overhead of large chunks with the load-balancing advantages of smaller dynamic chunks.